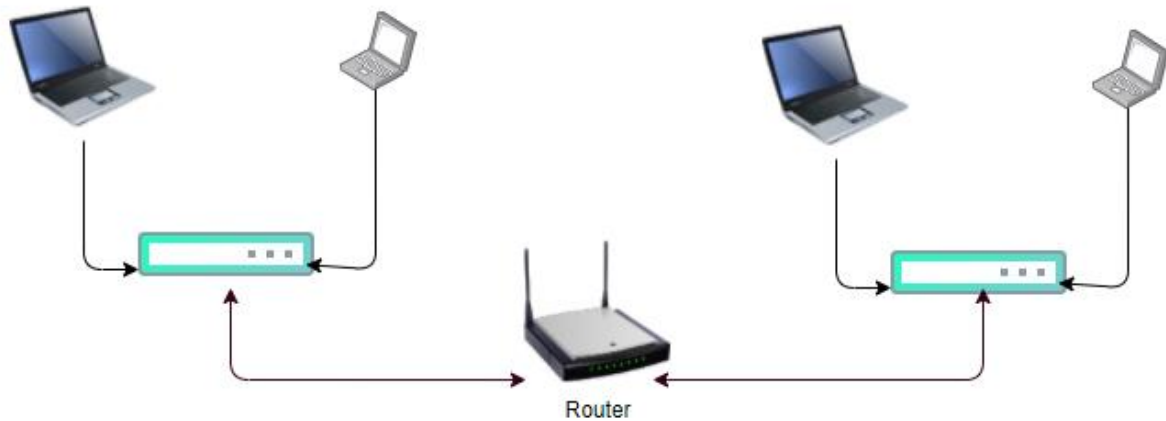


How networking Implements?



How networking Implements in K8s?

UC: suppose we have two nodes with in the same network, then both nodes can communicate with each-other using underlay network

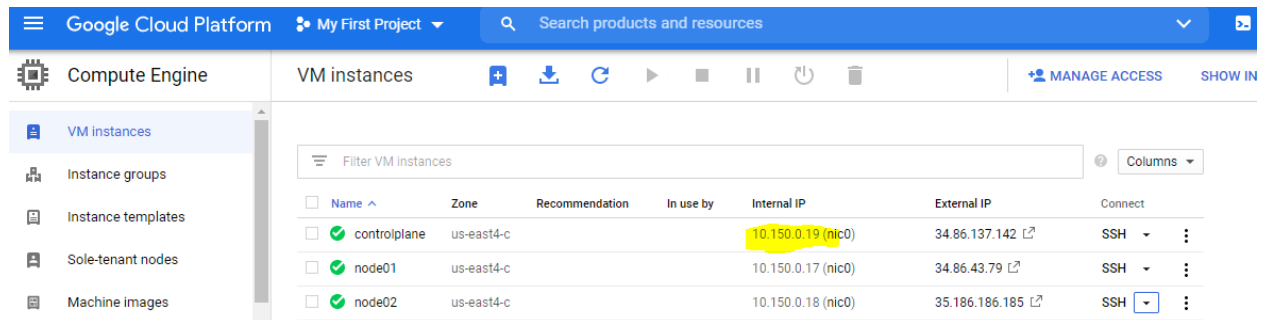


UC1: If not connected to any network

```
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 136 bytes 13892 (13.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 136 bytes 13892 (13.8 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

lo: LOOPBACK: 127.0.0.1

UC2: If connected to any network (GCP)



The screenshot shows the Google Cloud Platform interface for VM instances. The left sidebar contains navigation links for VM instances, instance groups, instance templates, sole-tenant nodes, and machine images. The main area displays a table of VM instances. The table has columns for Name, Zone, Recommendation, In use by, Internal IP, External IP, and Connect. Three instances are listed: controlplane, node01, and node02. The controlplane instance has an internal IP of 10.150.0.19 (nic0) and an external IP of 34.86.137.142. The node01 instance has an internal IP of 10.150.0.17 (nic0) and an external IP of 34.86.43.79. The node02 instance has an internal IP of 10.150.0.18 (nic0) and an external IP of 35.186.186.185. The Connect column shows SSH links for each instance.

Name	Zone	Recommendation	In use by	Internal IP	External IP	Connect
controlplane	us-east4-c			10.150.0.19 (nic0)	34.86.137.142	SSH
node01	us-east4-c			10.150.0.17 (nic0)	34.86.43.79	SSH
node02	us-east4-c			10.150.0.18 (nic0)	35.186.186.185	SSH

```
root@controlplane:~# ifconfig
ens4: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1460
    inet 10.150.0.19 netmask 255.255.255.255 broadcast 0.0.0.0
    inet6 fe80::4001:aff:fe96:13 prefixlen 64 scopeid 0x20<link>
    ether 42:01:0a:96:00:13 txqueuelen 1000 (Ethernet)
    RX packets 524 bytes 876937 (876.9 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 489 bytes 58366 (58.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 136 bytes 13892 (13.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 136 bytes 13892 (13.8 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

ens4: 10.150.0.19

UC3: After Installation of docker (pre-requisite for k8s)-docker0 created by docker

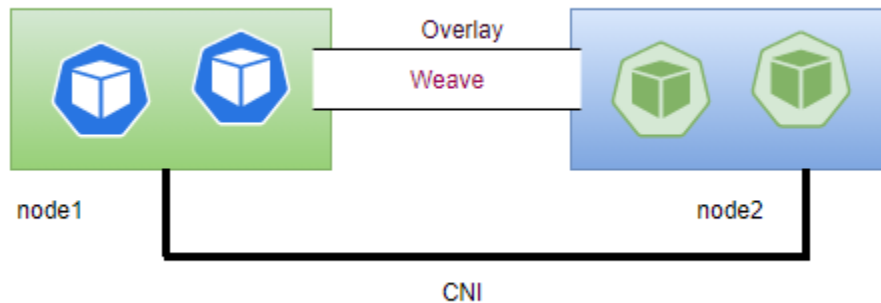
```
root@controlplane:~# docker -v
Docker version 20.10.2, build 2291f61
root@controlplane:~# ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:a1:38:80:90 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

ens4: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1460
    inet 10.150.0.19 netmask 255.255.255.255 broadcast 0.0.0.0
    inet6 fe80::4001:aff:fe96:13 prefixlen 64 scopeid 0x20<link>
    ether 42:01:0a:96:00:13 txqueuelen 1000 (Ethernet)
    RX packets 7121 bytes 127213503 (127.2 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 4796 bytes 459897 (459.8 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 276 bytes 29795 (29.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 276 bytes 29795 (29.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

docker0: 172.17.0.1

UC4: After Installation of CNI (Weave)



CNI runs agents on each machine, are responsible for assigning IP to Pod and Pod-Pod communication.

In Master

```
weave: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1376
    inet 10.32.0.1 netmask 255.240.0.0 broadcast 10.47.255.255
    inet6 fe80::2c80:e8ff:fe23:8a69 prefixlen 64 scopeid 0x20<link>
    ether 2e:80:e8:23:8a:69 txqueuelen 1000 (Ethernet)
    RX packets 156 bytes 14355 (14.3 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 139 bytes 34313 (34.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

weave: (subnet: 10.32.0.1)

In wnode01

```
weave: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1376
    inet 10.36.0.0 netmask 255.240.0.0 broadcast 10.47.255.255
    inet6 fe80::1e:94ff:fe90:8c prefixlen 64 scopeid 0x20<link>
    ether 02:1e:94:90:00:8c txqueuelen 1000 (Ethernet)
    RX packets 33 bytes 3822 (3.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 26 bytes 2362 (2.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

weave: (subnet: 10.36.0.0)

In wnode02

```
weave: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1376
    inet 10.44.0.0 netmask 255.240.0.0 broadcast 10.47.255.255
    inet6 fe80::485e:74ff:fe33:af07 prefixlen 64 scopeid 0x20<link>
    ether 4a:5e:74:33:af:07 txqueuelen 1000 (Ethernet)
    RX packets 49 bytes 4826 (4.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 26 bytes 2362 (2.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

weave: (subnet: 10.44.0.0)

Network solution (Weave) is responsible for creating a virtual switch (bridge network) on each node.

In case of Docker, docker0 take care of container-container communication but in case of K8s, underlying CNI (weave) is responsible for pod-pod communication.

If we see the same in wnode01 node it is in different subnet...

If we see the same in wnode02 node it is in different subnet...

Pod in wnode01 will get IP from subnet 10.36.0.0, Pod in wnode02 will get IP from 10.44.0.0

```
root@controlplane:~# kubectl create deploy webdep --image=lerndevops/alpine:sleep --replicas=5
deployment.apps/webdep created
root@controlplane:~# kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
webdep-7f6d6dcb79-7xnp6	1/1	Running	0	13s	10.44.0.1	node02	<none>	<none>
webdep-7f6d6dcb79-b7d5c	1/1	Running	0	13s	10.44.0.2	node02	<none>	<none>
webdep-7f6d6dcb79-nz965	1/1	Running	0	13s	10.36.0.1	node01	<none>	<none>
webdep-7f6d6dcb79-pqpdx	1/1	Running	0	13s	10.36.0.2	node01	<none>	<none>
webdep-7f6d6dcb79-rxqfz	1/1	Running	0	13s	10.36.0.3	node01	<none>	<none>

```
root@controlplane:~#
```

In this way K8s ensures that if we create a Pod, assigned the IP based on the weave subnet deployed on the node.

Can we assign our own IP addresses to the Pod?

Yes, while defining the CNI can assign own subnet range, each node has its own virtual switch.

UC: Pods with in same nodes are able to communicate to each other (within same subnet).

apt-get install iputils-ping

kubectl exec <pod> --ping <IP>

```
root@controlplane:~# kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
webdep-7f6d6dcb79-7xnp6	1/1	Running	0	4m7s	10.44.0.1	node02	<none>	<none>
webdep-7f6d6dcb79-b7d5c	1/1	Running	0	4m7s	10.44.0.2	node02	<none>	<none>
webdep-7f6d6dcb79-nz965	1/1	Running	0	4m7s	10.36.0.1	node01	<none>	<none>
webdep-7f6d6dcb79-pqpdx	1/1	Running	0	4m7s	10.36.0.2	node01	<none>	<none>
webdep-7f6d6dcb79-rxqfz	1/1	Running	0	4m7s	10.36.0.3	node01	<none>	<none>

```
root@controlplane:~# kubectl exec webdep-7f6d6dcb79-7xnp6 -- ping 10.44.0.2
PING 10.44.0.2 (10.44.0.2): 56 data bytes
64 bytes from 10.44.0.2: seq=0 ttl=64 time=0.259 ms
64 bytes from 10.44.0.2: seq=1 ttl=64 time=0.137 ms
64 bytes from 10.44.0.2: seq=2 ttl=64 time=0.141 ms
^C
root@controlplane:~# kubectl exec webdep-7f6d6dcb79-b7d5c -- ping 10.44.0.1
PING 10.44.0.1 (10.44.0.1): 56 data bytes
64 bytes from 10.44.0.1: seq=0 ttl=64 time=0.152 ms
64 bytes from 10.44.0.1: seq=1 ttl=64 time=0.128 ms
64 bytes from 10.44.0.1: seq=2 ttl=64 time=0.150 ms
10.44.0.164 bytes from 10.44.0.1: seq=3 ttl=64 time=0.142 ms
64 bytes from 10.44.0.1: seq=4 ttl=64 time=0.178 ms
```

kubectl exec <pod> --traceroute <IP>

```

root@controlplane:~# kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE   NOMINATED NODE   READINESS GATES
webdep-7f6d6dcb79-7xnp6            1/1    Running   0           8m8s  10.44.0.1       node02  <none>            <none>
webdep-7f6d6dcb79-b7d5c            1/1    Running   0           8m8s  10.44.0.2       node02  <none>            <none>
webdep-7f6d6dcb79-nz965            1/1    Running   0           8m8s  10.36.0.1       node01  <none>            <none>
webdep-7f6d6dcb79-pgpdx            1/1    Running   0           8m8s  10.36.0.2       node01  <none>            <none>
webdep-7f6d6dcb79-rxqfz            1/1    Running   0           8m8s  10.36.0.3       node01  <none>            <none>
root@controlplane:~# kubectl exec webdep-7f6d6dcb79-7xnp6 -- traceroute 10.44.0.2
traceroute to 10.44.0.2 (10.44.0.2), 30 hops max, 46 byte packets
 1  10.44.0.2 (10.44.0.2)  0.119 ms  0.008 ms  0.005 ms
root@controlplane:~# kubectl exec webdep-7f6d6dcb79-b7d5c -- traceroute 10.44.0.1
traceroute to 10.44.0.1 (10.44.0.1), 30 hops max, 46 byte packets
 1  10.44.0.1 (10.44.0.1)  0.011 ms  0.009 ms  0.005 ms
root@controlplane:~#

```

UC: Pods in wnode1 are trying to access the Pod in wnode2, able to access through the underlay network.

```

root@controlplane:~# kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE   NOMINATED NODE   READINESS GATES
webdep-7f6d6dcb79-7xnp6            1/1    Running   0           36m   10.44.0.1       node02  <none>            <none>
webdep-7f6d6dcb79-b7d5c            1/1    Running   0           36m   10.44.0.2       node02  <none>            <none>
webdep-7f6d6dcb79-nz965            1/1    Running   0           36m   10.36.0.1       node01  <none>            <none>
webdep-7f6d6dcb79-pgpdx            1/1    Running   0           36m   10.36.0.2       node01  <none>            <none>
webdep-7f6d6dcb79-rxqfz            1/1    Running   0           36m   10.36.0.3       node01  <none>            <none>
root@controlplane:~# kubectl exec webdep-7f6d6dcb79-7xnp6 -- ping 10.36.0.1
PING 10.36.0.1 (10.36.0.1): 56 data bytes
64 bytes from 10.36.0.1: seq=0 ttl=64 time=1.931 ms
64 bytes from 10.36.0.1: seq=1 ttl=64 time=0.540 ms
64 bytes from 10.36.0.1: seq=2 ttl=64 time=0.611 ms
64 bytes from 10.36.0.1: seq=3 ttl=64 time=0.723 ms

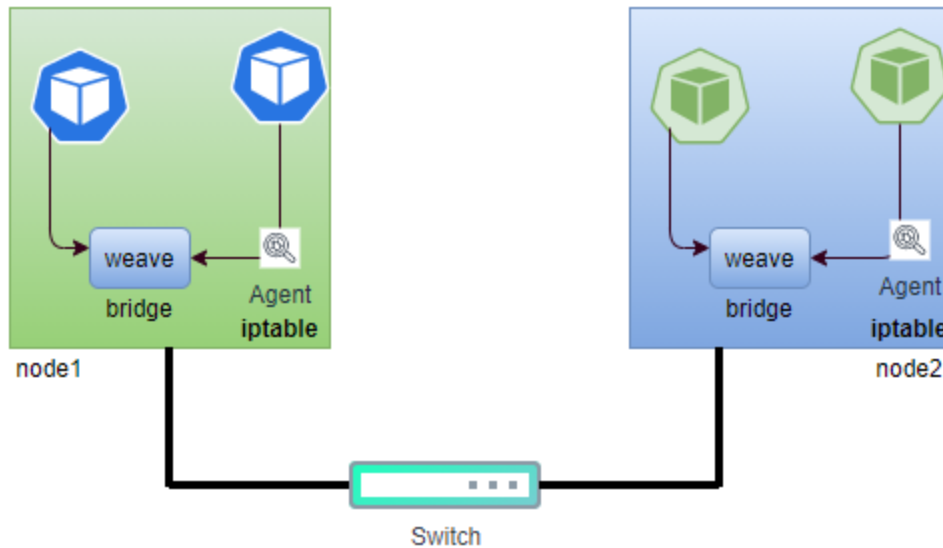
```

How communication between pods in different nodes using underlay network happen?

It's the weave network responsibility to route the response to Pod on other node this is going to use your underlay node network

On every node there is an iptable which allows to communicate among various nodes in the underlay network.

There is a route table created on every node, when you run the command route



There is a route table in each node.

```
root@controlplane:~# route
Kernel IP routing table
Destination    Gateway         Genmask         Flags Metric Ref    Use Iface
default        _gateway        0.0.0.0         UG    100    0      0 ens4
10.32.0.0      0.0.0.0         255.240.0.0     U      0      0      0 weave
_gateway       0.0.0.0         255.255.255.255 UH    100    0      0 ens4
172.17.0.0     0.0.0.0         255.255.0.0     U      0      0      0 docker0
root@controlplane:~#
```

There is a virtual network created by weave, there is a weave network agent running on each node as a pod, which is maintain all information of the Pod running on specific node.

```
root@controlplane:~# kubectl get pods -n kube-system -o wide
NAME                                READY    STATUS    RESTARTS   AGE    IP              NODE             NOMINATED NODE    READINESS GATES
ES
coredns-74ff55c5b-4rp15             1/1     Running   0           87m    10.32.0.3       controlplane     <none>             <none>
coredns-74ff55c5b-ksrck             1/1     Running   0           87m    10.32.0.2       controlplane     <none>             <none>
etcd-controlplane                   1/1     Running   0           87m    10.150.0.19     controlplane     <none>             <none>
kube-apiserver-controlplane         1/1     Running   0           87m    10.150.0.19     controlplane     <none>             <none>
kube-controller-manager-controlplane 1/1     Running   0           87m    10.150.0.19     controlplane     <none>             <none>
kube-proxy-cfr66                    1/1     Running   0           87m    10.150.0.19     controlplane     <none>             <none>
kube-proxy-gtgbm                    1/1     Running   0           73m    10.150.0.18     node02           <none>             <none>
kube-proxy-hcjwj                    1/1     Running   0           68m    10.150.0.20     node01           <none>             <none>
kube-scheduler-controlplane         1/1     Running   0           87m    10.150.0.19     controlplane     <none>             <none>
weave-net-8jcmx                     2/2     Running   1           68m    10.150.0.20     node01           <none>             <none>
weave-net-fzvtm                     2/2     Running   1           85m    10.150.0.19     controlplane     <none>             <none>
weave-net-zggjl                     2/2     Running   1           73m    10.150.0.18     node02           <none>             <none>
root@controlplane:~#
```

Weave is overlay network establish at cluster level, when we install the weave network, it's a **DaemonSet**

IP of master components

```
root@controlplane:~# kubectl get pods -n kube-system -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE               NOMINATED NODE   READINESS GATES
ES
coredns-74ff55c5b-4rp15             1/1     Running   0           71m   10.32.0.3       controlplane        <none>            <none>
coredns-74ff55c5b-ksrck             1/1     Running   0           71m   10.32.0.2       controlplane        <none>            <none>
etcd-controlplane                   1/1     Running   0           71m   10.150.0.19     controlplane        <none>            <none>
kube-apiserver-controlplane          1/1     Running   0           71m   10.150.0.19     controlplane        <none>            <none>
kube-controller-manager-controlplane 1/1     Running   0           71m   10.150.0.19     controlplane        <none>            <none>
kube-proxy-cfr66                    1/1     Running   0           71m   10.150.0.19     controlplane        <none>            <none>
kube-proxy-gtgbm                     1/1     Running   0           57m   10.150.0.18     node02              <none>            <none>
kube-proxy-hcjwj                     1/1     Running   0           51m   10.150.0.20     node01              <none>            <none>
kube-scheduler-controlplane          1/1     Running   0           71m   10.150.0.19     controlplane        <none>            <none>
weave-net-8jcmx                     2/2     Running   1           51m   10.150.0.20     node01              <none>            <none>
weave-net-fzvtm                     2/2     Running   1           68m   10.150.0.19     controlplane        <none>            <none>
weave-net-zggjl                     2/2     Running   1           57m   10.150.0.18     node02              <none>            <none>
root@controlplane:~#
```

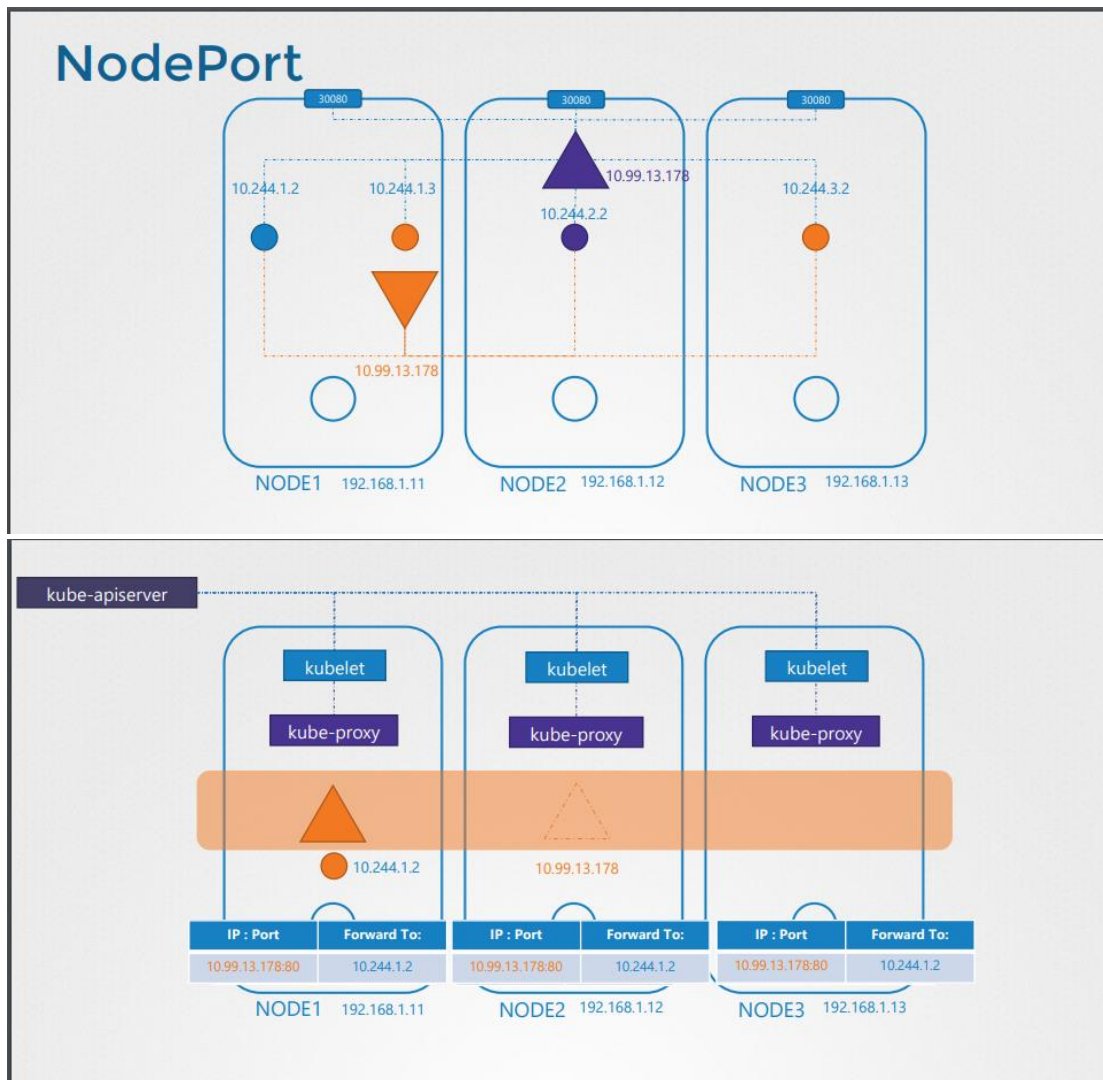
Master components joins host network

```
root@controlplane:~# kubectl get pods -n kube-system -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE               NOMINATED NODE   READINESS GATES
ES
coredns-74ff55c5b-4rp15             1/1     Running   0           71m   10.32.0.3       controlplane        <none>            <none>
coredns-74ff55c5b-ksrck             1/1     Running   0           71m   10.32.0.2       controlplane        <none>            <none>
etcd-controlplane                   1/1     Running   0           71m   10.150.0.19     controlplane        <none>            <none>
kube-apiserver-controlplane          1/1     Running   0           71m   10.150.0.19     controlplane        <none>            <none>
kube-controller-manager-controlplane 1/1     Running   0           71m   10.150.0.19     controlplane        <none>            <none>
kube-proxy-cfr66                    1/1     Running   0           71m   10.150.0.19     controlplane        <none>            <none>
kube-proxy-gtgbm                     1/1     Running   0           57m   10.150.0.18     node02              <none>            <none>
kube-proxy-hcjwj                     1/1     Running   0           51m   10.150.0.20     node01              <none>            <none>
kube-scheduler-controlplane          1/1     Running   0           71m   10.150.0.19     controlplane        <none>            <none>
weave-net-8jcmx                     2/2     Running   1           51m   10.150.0.20     node01              <none>            <none>
weave-net-fzvtm                     2/2     Running   1           68m   10.150.0.19     controlplane        <none>            <none>
weave-net-zggjl                     2/2     Running   1           57m   10.150.0.18     node02              <none>            <none>
root@controlplane:~# hostname -i
10.150.0.19
root@controlplane:~#
```

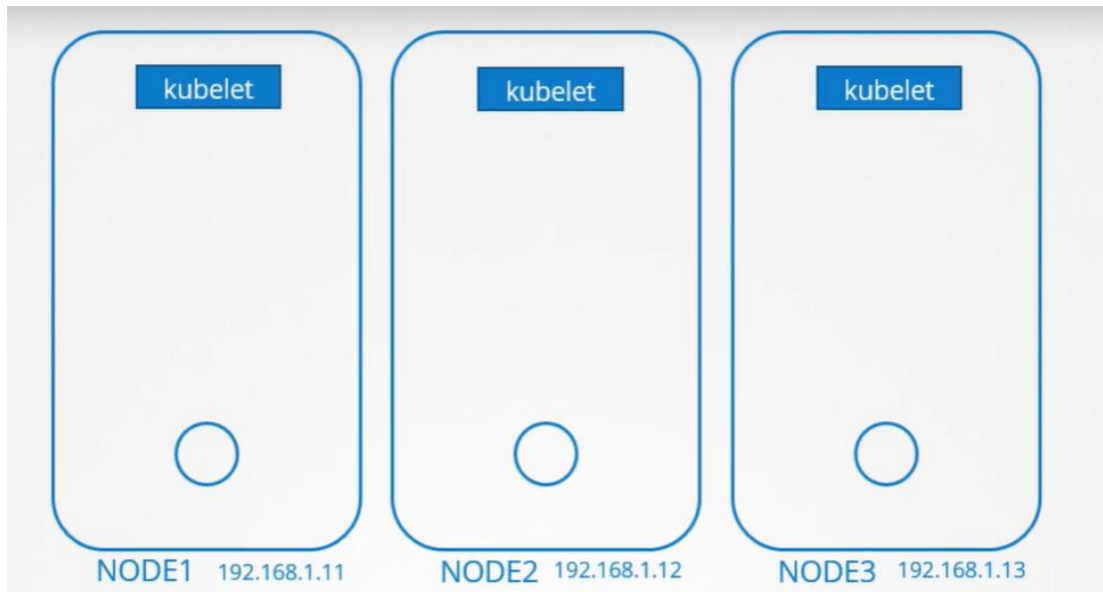
```
root@controlplane:~# hostname -i
10.150.0.19
root@controlplane:~# more /etc/kubernetes/manifests/kube-apiserver.yaml | grep -i host
- --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
  host: 10.150.0.19
  host: 10.150.0.19
  host: 10.150.0.19
  hostNetwork: true
- hostPath:
- hostPath:
- hostPath:
- hostPath:
- hostPath:
root@controlplane:~#
```

IP for nodes comes from cloud provider, not from weave.

Service Networking



You rarely configure pod to communicate directly with each other, pod communicate to each other through the service. Services are not bounded to specific node services, are always specific to cluster level .

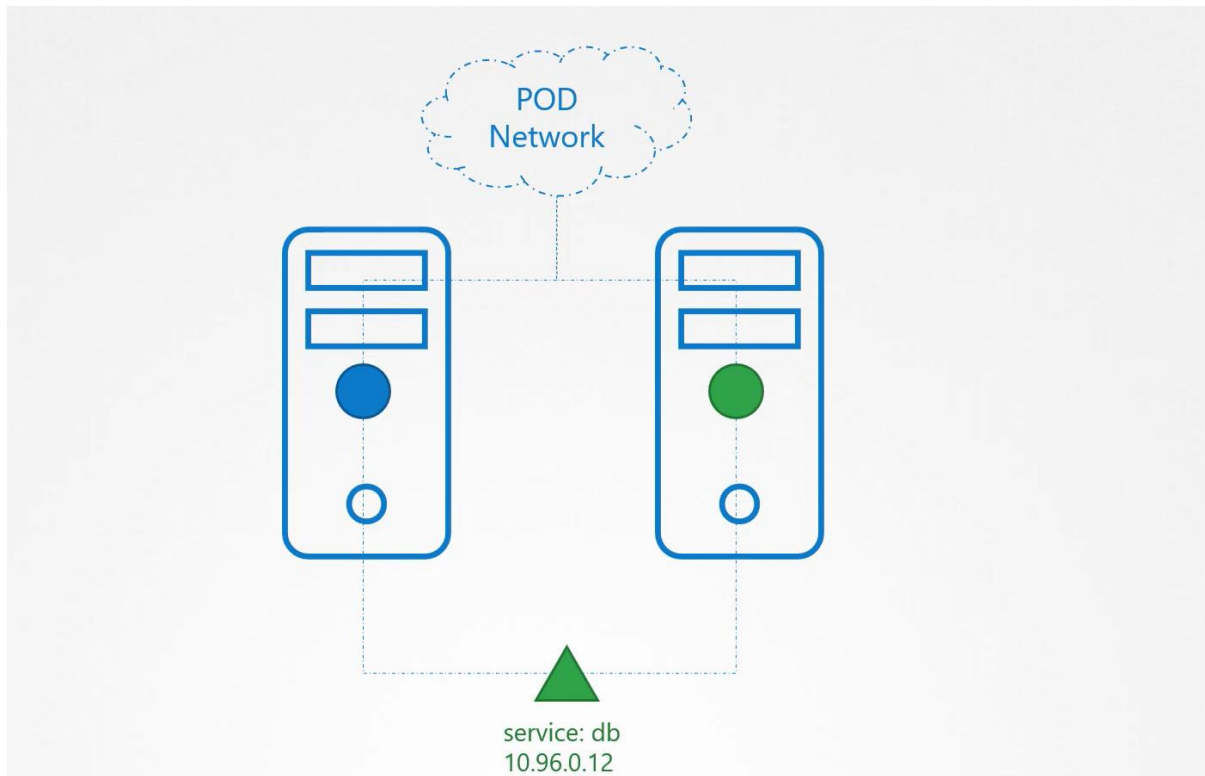


kubelet

kubelet is an agent run on each node of the cluster, its job is to look for new pod creation request from api-server, forward the request to runtime engine. When pod is created, it works with CNI network and to generate the ip, ip address generated by weave network assigned to the pod and at same time, update to apiserver and api-server is going to update etcd db, going to **register with default core dns server**. Pod can be reached out using its ip address or its dns name.

kube-proxy

- kube-proxy is an agent run on each node of the cluster, its job is to watches the kubernetes control plane for services and endpoints, for each service it installs iptable rules in each node of the cluster.
- Iptable rules capture service's ClusterIP and port, and redirect that traffic to one of the service's backend sets.
- For each endpoint object, it installs iptable rules which select a backend pod.
- kube-proxy relies on readiness probe to determine if a backing pod is healthy.
- When a service is created a DNS entry automatically created in DNS server.



In above case web pod can access the db-service using the db-service name, service also get an IP address assigned to it, whenever web pod tries to reach the service using its IP or name, it forward the traffic to backend Pod, in this case the database.

UC: What is the service and how it gets an IP address?

Does the service join the same Pod network? The service can't join the Pod network, because the service is not an actual thing it is not a container like Pods, so it does not have an interface or an actively listening process it is a virtual component that only lives in K8s memory, service is also accessible from any node of the cluster, **so how it is achieved?**

Here kube-proxy comes in

kube-proxy is a process run on each node of the cluster, its job is to look for new services and every time a new service is created, it creates an appropriate rule on each node, to forward the traffic to those services to the backend pods, one way of do this using IP table rules.